A Major Project Report

On

Web based Graphical Password Authentication System

Submitted to Jawaharlal Nehru Technological University, Hyderabad, in partial fulfillment of the requirements for the award of the degree in

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

Dage Krishna varsha	**-22PP1A0514**
Dandaboina Bindu	**-22PP1A0515**
Dandugula Poojitha	**-22PP1A0516**
Dasari Anitha	**-22PP1A0517**

ANUBOSE INSTITUTE OF TECHNOLOGY FOR WOMENS

UGC Autonomous Institution

Approved by AICTE New Delhi, Affiliated to JNTUH. Accredited by NAAC with "B++" Grade

New Paloncha, Bhadradri Kothagudem, Telangana

2025-26

ANUBOSE INSTITUTE OF TECHNOLOGY FOR WOMENS

UGC Autonomous Institution

Approved by AICTE New Delhi, Affiliated to JNTUH. Accredited by NAAC with "B++" Grade

New Paloncha, Bhadradri Kothagudem, Telangana

CERTIFICATE

This is to certify that the project report entitled **"Web based Graphical Password Authentication System"** being submitted by

Dage Krishna varsha	-22PP1A0514
Dandaboina Bindu	-22PP1A0515
Dandugula Poojitha	-22PP1A0516
Dasari Anitha	-22PP1A0517

in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, Jawaharlal Nehru Technological University Hyderabad, is a record of Bonafide work carried out under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or institute for the award of any Degree.

Internal Guide

Mr. Y.Mukund Reddy

Assistant Professor

Head of the Department

Dr. D. Praneeth Ph.D

Assistant Professor

External Examiner

Principal

DECLARATION

We declare that this project report "**Web based Graphical Password Authentication System"** submitted in partial fulfillment of the degree of B. Tech in Computer Science and Engineering is a record of original work carried out by us under the guidance of**. Mr.Mukund Reddy,** Assistant Professor and was not submitted to any other Institution or University for award of any other degree.

Project Associates:

Dage Krishna varsha	-22PP1A0514
Dandaboina Bindu	-22PP1A0515
Dandugula Poojitha	- 22PP1A0516
Dasari Anitha	-22PP1A0517

Place:

Date:

ACKNOWLEDGEMENT

We express our deep sense of gratitude and thanks to our project guide **Mr.Y.Mukund Reddy, Assistant Professor** of Computer Science and Engineering under whose guidance and supervision this work had been accomplished.

We are deeply indebted to our Head of the Department **Dr.D. Praneeth, Ph.D. Assistant Professor** who modeled us both technically and morally for achieving greater success in life.

We are very grateful to our Principal **Dr. A. Avani, Ph.D. Professor** for providing us with an environment to complete our project successfully.

We deeply thank Our Chairman & correspondent, **Dr. T. Bharath Krishna M. Tech. Ph. D**, with whose support and provision this project has been carried out with required infrastructure.

We are thankful to the **Internal Department Committee** who have invested their valuable time to conduct our monthly presentations and provided their feedback with a lot of useful suggestions We also thank all the **faculty members**, Department of Computer Science & Engineering for their encouragement and assistance.

Project Associates:

Dage Krishna varsha -22PP1A0514
Dandaboina Bindu -22PP1A0515
Dandugula Poojitha - 22PP1A0516
Dasari Anitha -22PP1A0517

ABSTRACT

With the rapid growth of web-based applications, ensuring secure authentication has become increasingly important. Traditional text-based password systems are widely used but suffer from several security issues such as weak password selection, brute-force attacks, and shoulder surfing. To address these challenges, this project proposes a **Web-Based Graphical Password Authentication System** that enhances security by combining graphical and textual authentication techniques. In this system, users register by uploading an image and selecting specific points (spots) on that image, which act as their password. During login, users must correctly identify the same points to gain access. This approach improves security and usability, as graphical passwords are easier to remember and harder to observe or replicate compared to traditional text-based passwords.

The system is implemented using Python (Django framework) and MySQL database, with several key algorithms supporting its functionality. These include a **Coordinate Extraction Algorithm** to capture pixel values from user clicks, a **Tolerance-Based Matching Algorithm** that allows slight variations (±10 pixels) for user convenience, and a **Spot Validation Algorithm** to verify whether selected points match stored values. Additionally, a **Sequential Multi-Point Authentication Algorithm** ensures that all selected spots are validated in order, while a **Database Retrieval and Matching Algorithm** compares user input with stored credentials. Together, these algorithms provide a secure and efficient authentication mechanism that minimizes risks such as shoulder surfing and unauthorized access, making the system a reliable alternative to conventional password-based authentication methods.

Contents **Page No**

Figures **PAGE NO**

Tables and Abbreviations

Tables: **Page No.**

Abbreviations:

AI : Artificial Intelligence

ANN: Artificial Neural Network

API : Application Programming Interface

CNN: Convolutional Neural Network

CSV: Comma Separated Values

CSS: Cascading Style Sheets

DB: Database

DFD: Data Flow Diagram

HTML: Hyper Text Markup Language

IDE: Integrated Development Environment

JS: JavaScript

KNN: K-Nearest Neighbors

ML: Machine Learning

MVT: Model View Template

NLP: Natural Language Processing

OS: Operating System

RNN: Recurrent Neural Network

SVM: Support Vector Machine

SQL: Structured Query Language

UML: Unified Modeling Language

UAT: User Acceptance Testing

UI: User Interface

URL: Uniform Resource Locator

Web based Graphical Password Authentication System

CHAPTER – 1

1. INTRODUCTION

In today's digital world, user authentication plays a vital role in protecting sensitive information and ensuring secure access to systems and applications. With the rapid increase in web-based services such as online banking, social media, cloud storage, and e-commerce platforms, the need for reliable and secure authentication mechanisms has become more critical than ever.

One of the major problems with text-based passwords is that users tend to choose weak and easily guessable passwords for the sake of convenience. Many users reuse the same password across multiple platforms, increasing the risk of unauthorized access if one account is compromised.

The proposed system, **Web-Based Graphical Password Authentication System**, utilizes an image-based approach combined with coordinate-based validation. During registration, users upload an image and select multiple points (spots) on that image as their password.

From a technical perspective, the system is implemented using Python with a web framework (Django), along with a MySQL database for storing user credentials.

In conclusion, the increasing demand for secure authentication systems necessitates innovative solutions beyond traditional passwords. The graphical password authentication system presented in this project offers an effective alternative by combining visual memory with advanced validation techniques.

CHAPTER-2

1. LITERATURE SURVEY

Authentication systems have evolved significantly over the years to address increasing security threats in digital environments. Traditional text-based password systems have been the most commonly used method for user authentication due to their simplicity and ease of implementation.

To improve password security, various approaches have been proposed, including enhanced textual password mechanisms. Wantong Zheng and Chunfu Jia introduced the **Combined Password (Combined PWD)** method, which incorporates separators such as spaces within passwords to increase complexity.

Another approach to strengthening authentication is the use of **time-based dynamic passwords**. Researchers have proposed systems where passwords change over time based on predefined rules set by the user.

Graphical password authentication has emerged as a promising alternative to traditional methods. Research indicates that humans have a superior ability to remember images compared to text.

Hua Wang and Yao Guo proposed the **Desktop Password Authentication Center (DPAC)**, a reuse-oriented framework that allows sharing of security mechanisms across multiple applications. This approach reduces redundancy and enhances overall system security. Similarly, Salah Refish introduced the **Password Authentication Code (PAC)** scheme, which focuses on secure communication between users during authentication.

CHAPTER – 3

2. SYSTEM ANALYSIS

System analysis is an essential phase in software development that focuses on understanding the existing system, identifying its limitations, and proposing an improved solution. In this project, the analysis is carried out to evaluate the drawbacks of traditional authentication methods and to design a more secure and user-friendly graphical password authentication system.

2.1 EXISTING SYSTEM

The existing system primarily relies on **text-based password authentication**, where users enter a username and password to access a system. This method is widely used across various applications such as websites, banking systems, and enterprise platforms due to its simplicity and ease of implementation.

Although text-based authentication is convenient, it heavily depends on user behavior. Many users tend to choose simple and easy-to-remember passwords, such as common words, names, or predictable patterns.

2.2 DISADVANTAGES OF EXISTING SYSTEM

The existing text-based authentication system has several limitations:

- **Weak Password Selection:** Users often choose simple passwords that are easy to guess.
- **Password Reuse:** The same password is used across multiple accounts, increasing risk.
- **Poor Memorability:** Strong passwords are difficult to remember, leading to insecure storage practices.

2.3 PROPOSED SYSTEM

To overcome the limitations of the existing system, a **Web-Based Graphical Password Authentication System** is proposed. This system introduces a more secure and user-friendly authentication mechanism by combining graphical and textual elements.

In the proposed system, users register by uploading an image and selecting specific points (spots) on the image as their password.

The system uses a **tolerance-based approach**, allowing slight variations in user input (±10 pixels) to improve usability. It also implements a **multi-level authentication process**, where both username verification and graphical password validation are required

2.4 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over traditional methods:

2.4.1 **Improved Security:** Harder to guess or replicate graphical passwords.

2.4.2 **Resistance to Shoulder Surfing:** Difficult for attackers to observe exact click points.

2.4.3 **Better Memorability:** Images are easier to remember than complex text passwords.

2.4.4 **Tolerance-Based Input:** Allows small variations, improving user experience.

2.4.5 **Multi-Level Authentication:** Adds an extra layer of security.

2.5 OBJECTIVES OF THE PROJECT

The main objectives of this project are:

1. **To develop a secure authentication system** that overcomes the limitations of traditional text-based passwords.
2. **To implement graphical password techniques** using image-based point selection for enhanced security.

3. **To design a scalable and efficient system** using modern web technologies and database management.

4. FEASIBILITY STUDY

Feasibility study is an important phase in system development that determines whether the proposed system is practical, viable, and beneficial. The feasibility of the proposed **Web-Based Graphical Password Authentication System** is analyzed based on the following three key factors:

1. ECONOMICAL FEASIBILITY

Economic feasibility refers to the cost-effectiveness of the proposed system. It analyzes whether the benefits of the system outweigh the costs involved in its development and implementation. In this project, the development cost is minimal because most of the tools and technologies used, such as Python, Django framework, and MySQL database, are open-source and freely available.

1. TECHNICAL FEASIBILITY

Technical feasibility evaluates whether the system can be developed and implemented using the available technology and resources. The proposed system is built using widely used technologies such as Python, Django, HTML, CSS, and MySQL, which are reliable and well-supported.

1. SOCIAL FEASIBILITY

Social feasibility refers to the level of acceptance of the system by users and its impact on society. The proposed graphical password system is user-friendly and easy to understand, as it uses images instead of complex textual passwords.

The system enhances security, which increases user trust and confidence. Users do not feel threatened by the system, as it simplifies the authentication process while maintaining privacy.

Minimal training is required for users to adapt to the system.

CHAPTER – 4

SYSTEM REQUIREMENTS

System requirements define the necessary resources and functionalities needed to develop and operate the proposed system. These requirements are categorized into software requirements, hardware requirements, functional requirements, and non-functional requirements.

4.1 SOFTWARE REQUIREMENTS

The software requirements specify the tools and technologies used for developing and running the system. The proposed **Web-Based Graphical Password Authentication System** uses the following software components:

- **Operating System:** Windows 10 or above
- **Programming Language:** Python
- **Framework:** Django
- **Frontend Technologies:** HTML, CSS, JavaScript
- **Database:** MySQL
- **Development Tool/IDE:** PyCharm / VS Code
- **Web Server:** Django development server (or Apache for deployment)
- **Browser:** Google Chrome / Mozilla Firefox

These tools are widely used, open-source, and provide a reliable platform for developing secure web applications.

4.1 HARDWARE REQUIREMENTS

The hardware requirements define the minimum system specifications needed to run the application efficiently:

- **Processor:** Intel i3 or higher
- **RAM:** 4 GB or above
- **Hard Disk:** 20 GB free space
- **Monitor:** 15-inch LED or higher

4.2 FUNCTIONAL REQUIREMENTS

Functional requirements describe the core functionalities that the system must perform:

- **User Registration:**
  The system should allow users to register by providing details such as username, email, and uploading an image for graphical password selection.

- **Graphical Password Creation:**
  Users should be able to select multiple points (spots) on the uploaded image, which will be stored as their password.

- **User Login:**
  The system should authenticate users by verifying the selected image points along with the username.

- **Tolerance-Based Validation:**
  The system should allow a small tolerance range (±10 pixels) while matching selected points to improve usability.

- **Password Reset:**
  Users should be able to reset their graphical password by uploading a new image and selecting new points.

4.1 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and performance characteristics of the system:

- **Security:**
  The system must protect user data and prevent unauthorized access using secure authentication techniques.

- **Usability:**
  The system should be user-friendly, with simple interfaces for registration and login.

- **Performance:**
  The system should respond quickly to user requests with minimal delay.

- **Reliability:**
  The system should function consistently without failures or errors.

- **Scalability:**
  The system should handle an increasing number of users without performance degradation.

- **Availability:**
  The system should be accessible whenever required with minimal downtime.

CHAPTER – 5

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, and data flow of the proposed system. It provides a clear understanding of how the system operates and how different components interact with each other. The design of the **Web-Based Graphical Password Authentication System** ensures security, efficiency, and user-friendliness.

4.1 SYSTEM ARCHITECTURE

The proposed system follows a **client-server architecture** combined with the **Model-View-Controller (MVC)** design pattern. In this architecture, the user interacts with the system through a web interface (client), while the server processes requests, performs authentication, and communicates with the database.

- **Client Side:**
  The user accesses the system through a web browser. The interface allows users to register, upload images, select graphical password points, and log in.

- **Server Side:**
  The server handles user requests, processes authentication logic, and manages communication between the user interface and the database.

- **Database:**
  The MySQL database stores user information such as username, image password, and selected coordinate points.

- **MVC Architecture:**

  - **Model:** Manages data and database operations.

  - **View:** Handles user interface and display of information.

  - **Controller:** Processes user input and controls application flow.

This architecture ensures modularity, scalability, and efficient data handling.

4.2 MODULES

1. Public Module

This module provides general access to the system. Any user can access the home page and navigate through available options such as registration and login.

2. User Registration Module

This module allows new users to create an account. Users enter their personal details and upload an image. They then select multiple points on the image, which are stored as their graphical password.

3. User Login Module

This module authenticates users. The system displays the uploaded image, and users must select the correct points. The system verifies these points using stored data and grants access if they match.

4. Password Reset Module

This module allows users to update their graphical password. Users can upload a new image and select new points for authentication.

4.3 SYSTEM DESIGN

System design describes how data flows through the system and how different components interact.

1. Data Flow Design (DFD)

The Data Flow Diagram represents how data moves within the system.

- Users provide input during registration and login.
- The system processes the data and stores or retrieves it from the database.

1. UML DESIGN

Unified Modeling Language (UML) diagrams are used to represent the system visually:

- **Use Case Diagram:**
  Shows interactions between users and the system, such as registration, login, and password reset.

- **Class Diagram:**
  Represents system classes, their attributes, and relationships.

- **Sequence Diagram:**
  Shows the sequence of interactions between system components during authentication.

- **Activity Diagram:**
  Represents the workflow of activities such as user login and validation process.

1. SYSTEM ARCHITECTURE

The architecture chooses how the framework should work. Request response time, page loading time, Ability to deal with the various requests, and so on are characterized by the design of the web application. In this manner, for better execution, it is indeed to utilize the best design. Here it utilizes the MVC architecture (Model-View-Control Architecture).

MVC Architecture implies Model-View-Controller architecture, which is an example architecture plan for programming projects.

Fig. 2. MVC architecture

The design has 3 parts, they are Model, View, and Controller (Fig 3). These segments make the framework more adjustable.

1.System Architecture

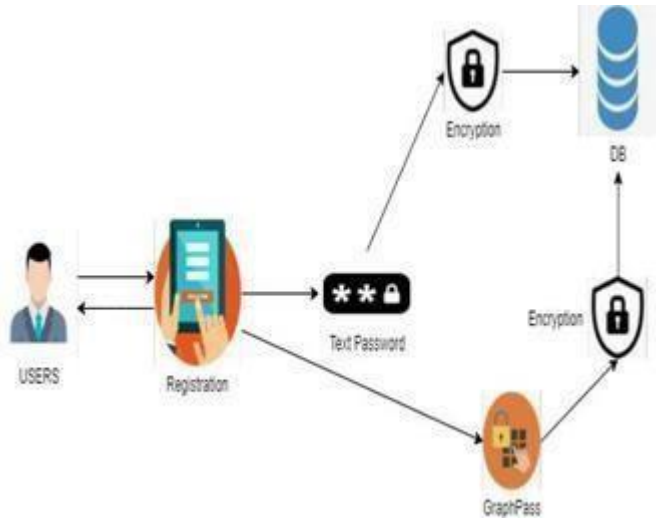


Fig. 3. System Architecture

On the border of the client, the user requests the registration. The Registration process includes two encryptions. One for text password, other for Graphical Password. Graph Pass was divided into 4 slices. Encryption takes place in each slice. The user-friendly graphical user interfaces make the task easier. Accordingly, the client doesn't have to think about the programming language and ideas.

The framework strictly follows the rules of Model view controller design (MVC architecture). MVC Architecture implies Model-View-Controller architecture, which is an example architecture plan for programming projects.

2. IMPLEMENTATION Tools used for the implementation are:

1.Software tools

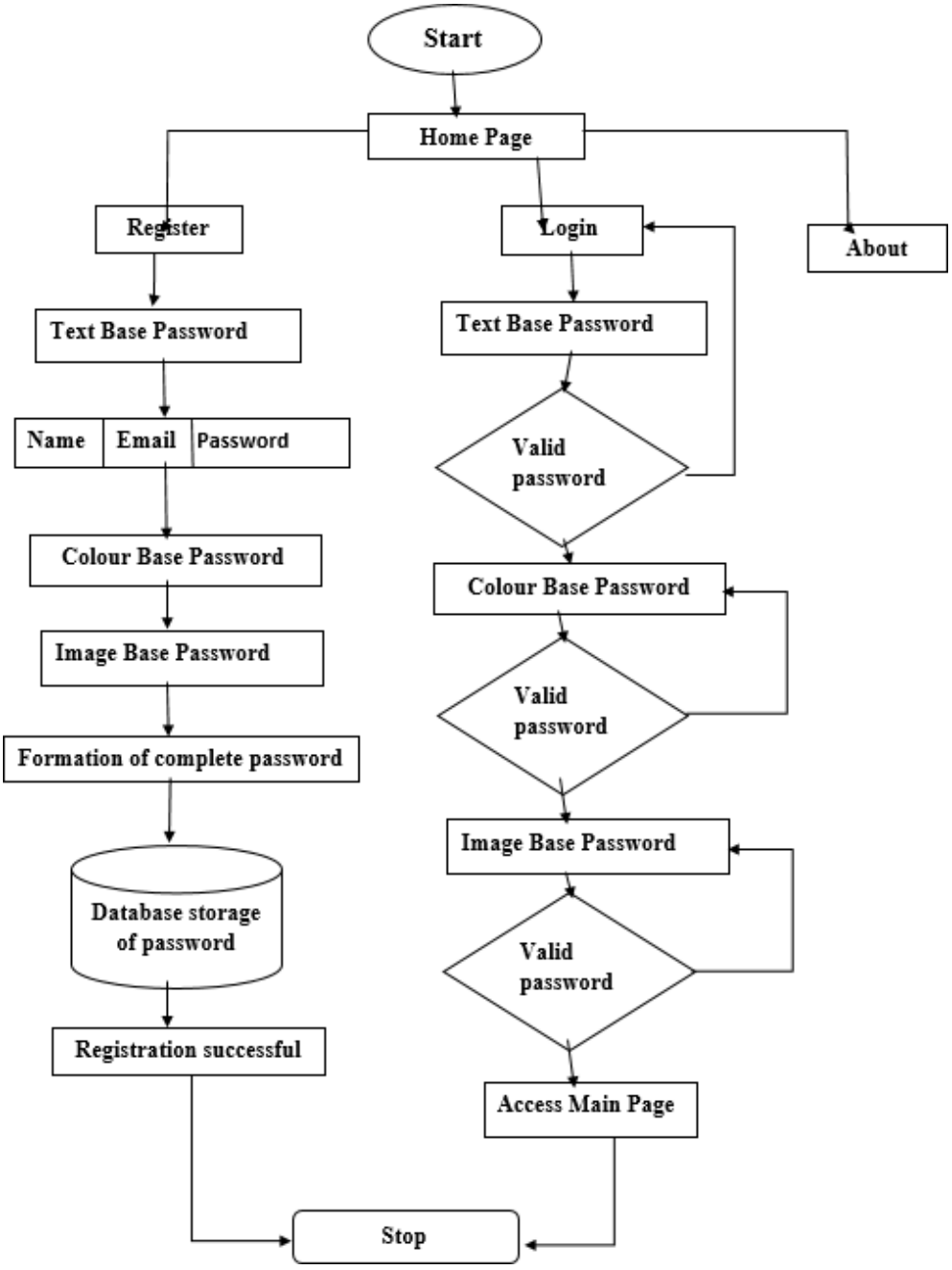
The text editor used for this development is sublime text. Sublime Text is a shareware cross-platform source code editor with a Python application programming interface (API). It natively supports many programming languages and markup languages, and functions can be attached by users with plugins, typically community-built and maintained under free-software licenses.

Hardware tools

Hardware requirements for this development are an i3+ processor, 4GB+ Ram, and 2GB+ SSD space.

6.2 DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.


Start
Home Page
Register
Login
About
Text Base Password
Name
Email
Password
Colour Base Password
Image Base Password
Formation of complete password
Database storage of password
Registration successful
Text Base Password
Valid password
Colour Base Password
Valid password
Image Base Password
Valid password
Access Main Page
Stop

UML DIAGRAMS:

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

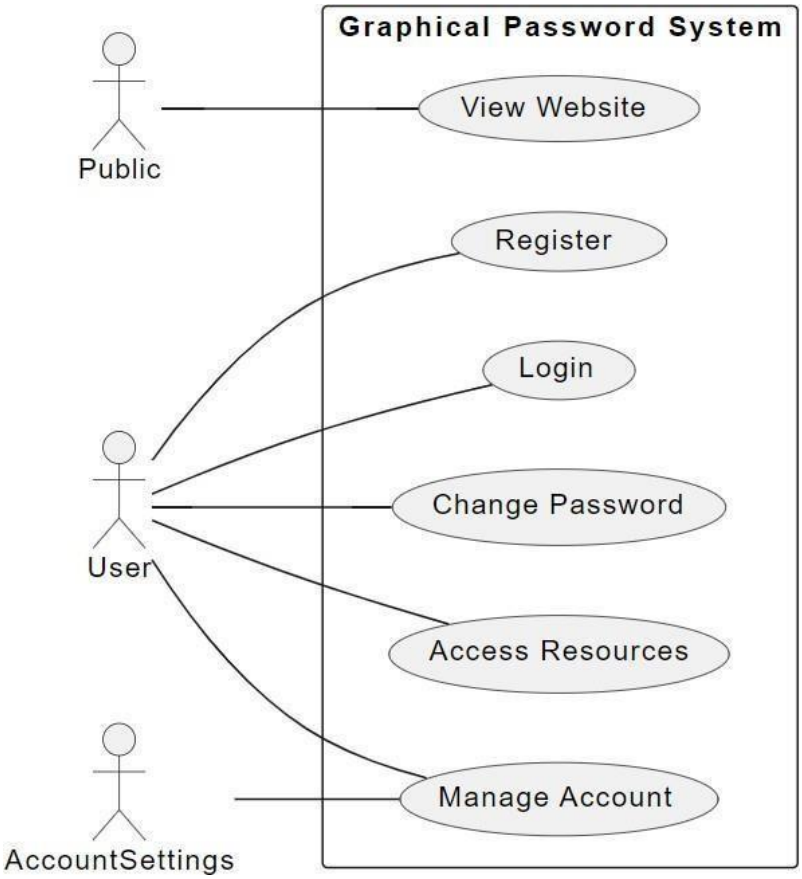
GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.

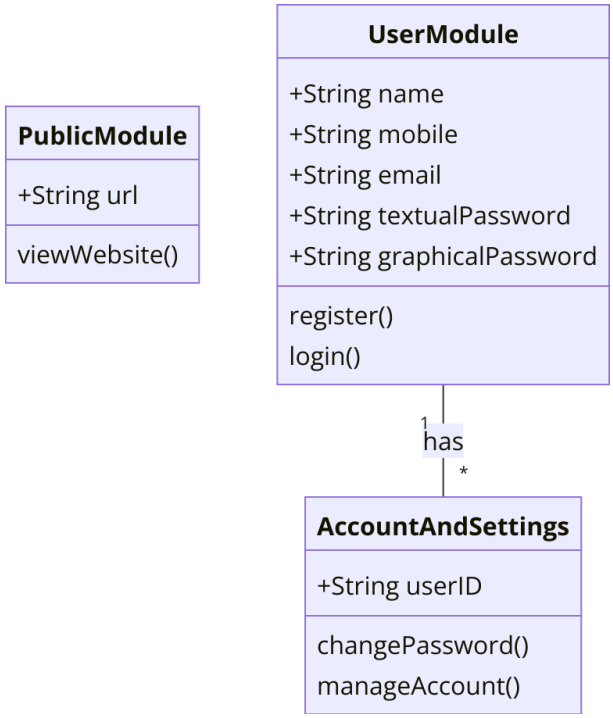
USE CASE DIAGRAM:

1. A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases.



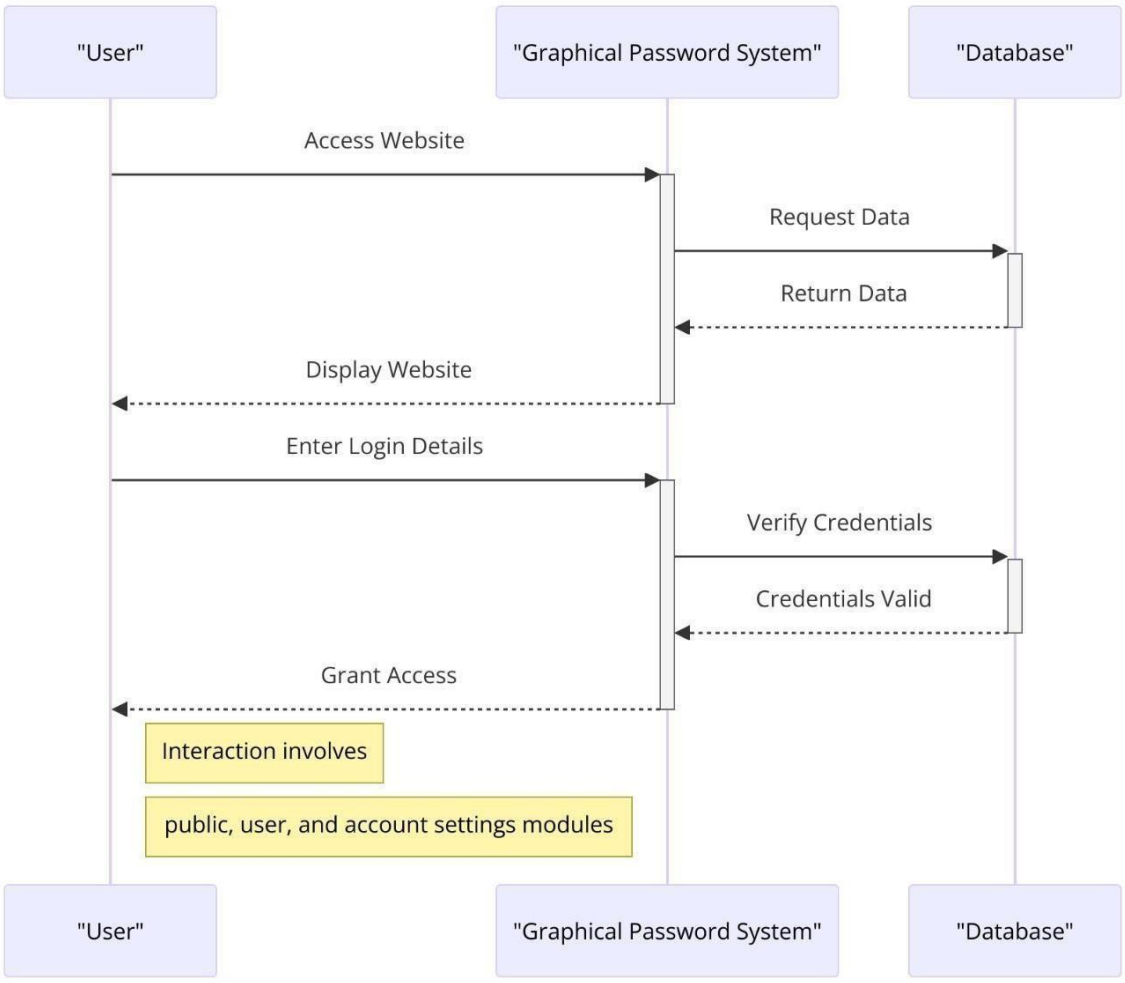
CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



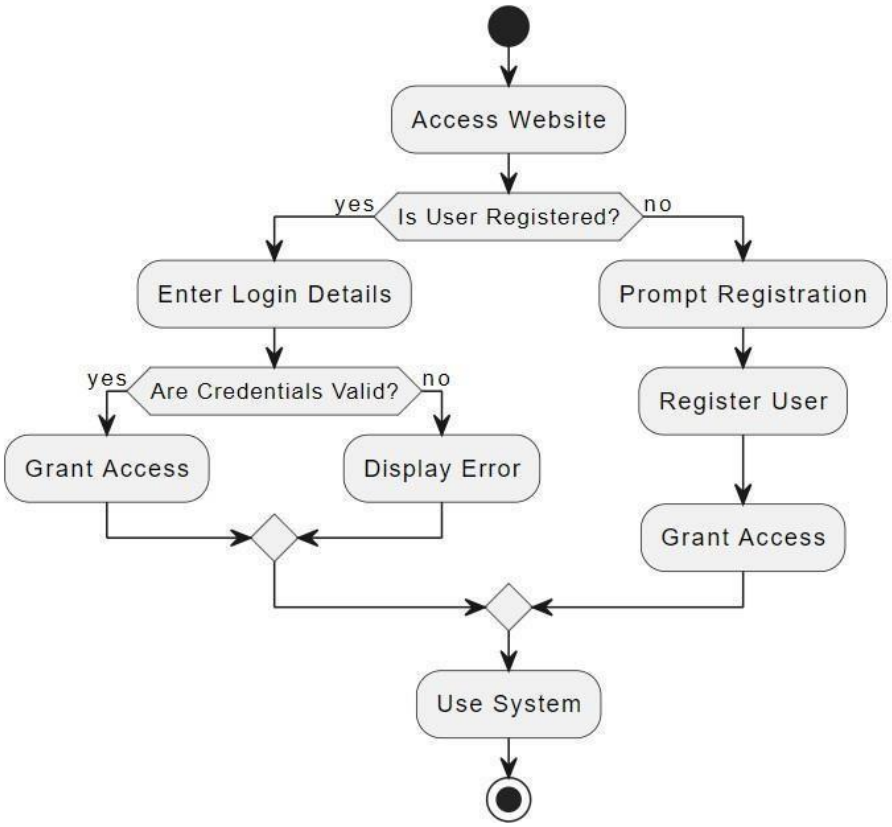
SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



CHAPTER – 6

METHODOLOGY AND IMPLEMENTATION

The methodology describes the step-by-step process followed to design and develop the **Web-Based Graphical Password Authentication System**. The system is implemented using a structured approach to ensure security, efficiency, and usability. It follows a client-server model where users interact through a web interface, and the server processes authentication requests.

1. METHODOLOGY

- **Requirement Analysis:**
  In this phase, the limitations of traditional password systems are analyzed, and the need for a graphical authentication system is identified.

- **System Design:**

  The architecture, modules, and data flow of the system are designed using MVC architecture.

- **Development:**

  The system is implemented using Python (Django framework), HTML, CSS, and MySQL database. Backend logic is developed to handle user authentication and data storage.

- **Testing:**

  The system is tested using various testing methods such as unit testing, integration testing, and functional testing to ensure correctness and reliability.

- **Deployment:**

  The final system is deployed on a web server, making it accessible to users through a browser.

- **2. IMPLEMENTATION**

User Registration

- The user enters personal details and uploads an image.
- The system displays the image and allows the user to select multiple points (spots).
- The selected coordinates are stored in the database along with user details.

User Login

- The user enters the username.
- The system retrieves and displays the corresponding image.

Password Reset

- The user uploads a new image.
- New coordinate points are selected and updated in the database.

Admin Module

- Admin logs into the system.
- Admin can view all registered users and their details.

6.1 ALGORITHMS USED IN THE PROJECT

The system uses several algorithms to ensure secure and accurate authentication:

1. Coordinate Extraction Algorithm

Purpose:
To extract the X and Y coordinates of points selected by the user on the image.

Steps:

1. Capture mouse click position on the image.
2. Read the pixel coordinates (X, Y).
3. Store the coordinates in a structured format.
4. Save the values in the database.

2. Tolerance-Based Matching Algorithm

Purpose:
To allow slight variations in user input during login.

Steps:

1. Retrieve stored coordinates (X_1, Y_1).
2. Capture user input coordinates (X_2, Y_2).
3. Define tolerance range (±10 pixels).
4. Check if:
    - X_2 is within (X_1 - 10) to (X_1 + 10)

- Y_2 is within $(Y_1 - 10)$ to $(Y_1 + 10)$

5. If both conditions are satisfied, consider it a match.

3. Spot Validation Algorithm

Purpose:
To verify whether selected points match stored password points.

Steps:

1. Retrieve stored spots from the database.
2. Compare each selected spot with stored values.
3. Apply tolerance-based matching.

4. Sequential Multi-Point Authentication Algorithm

Purpose:
To ensure all selected points are validated in sequence.

Steps:

1. Accept multiple input points from the user.
2. Validate each point one by one.
3. Maintain sequence order.

5. Database Retrieval and Matching Algorithm

Purpose:

To retrieve and verify user credentials from the database.

Steps:

4. Accept username input.
5. Fetch corresponding user data from the database.
6. Retrieve stored image and coordinates.

CHAPTER – 7

SOFTWARE INTRODUCTION, TESTING AND SOURCE CODE

This chapter describes the software tools used in the project, testing methods applied to ensure system reliability, and the implementation of the source code.

7.1 SOFTWARE INTRODUCTION

The development of the **Web-Based Graphical Password Authentication System** is carried out using modern and reliable software technologies. These tools ensure efficient development, security, and scalability of the system.

1. Python

Python is a high-level, interpreted programming language known for its simplicity and readability. It is widely used for web development, data processing, and automation. In this project, Python is used to implement backend logic, authentication processes, and database interactions.

2. Django Framework

Django is a powerful Python web framework that follows the Model-View-Controller (MVC) architecture (commonly referred to as MTV in Django). It provides built-in features such as authentication support, URL routing, and database handling, making development faster and more secure.

3. MySQL Database

MySQL is a relational database management system used to store user data securely. It stores information such as username, password image, and selected coordinate points. It ensures efficient data retrieval and management.

4. HTML, CSS, and JavaScript

These technologies are used for designing the user interface.

- HTML structures the web pages
- CSS enhances visual appearance
- JavaScript handles client-side interactions such as capturing mouse click positions

5. Development Tools

Tools like PyCharm or Visual Studio Code are used for coding and debugging. These tools provide features such as syntax highlighting, error detection, and project management.

7.2 SOFTWARE TESTING

Software testing is performed to ensure that the system works correctly and meets user requirements. It helps identify errors and improve system reliability.

1. Unit Testing

Each module of the system is tested individually to verify that it functions correctly. For example, registration, login, and password validation modules are tested separately.

2. Integration Testing

This testing ensures that different modules work together properly. For instance, interaction between the user interface, backend logic, and database is tested.

3. Functional Testing

The system is tested against functional requirements:

- Valid inputs are accepted
- Invalid inputs are rejected
- All system functions operate correctly

4. System Testing

The complete system is tested as a whole to ensure it meets all requirements and performs as expected.

5. User Acceptance Testing

End users test the system to verify usability and performance. Feedback is collected to ensure the system is user-friendly.

Test Results

All modules were tested successfully. The system performs efficiently with accurate authentication and minimal errors.

Sample Code

```
def UserLogin(request):
    global username
    if request.method == 'POST':
        username = request.POST.get('t1', False)
        password = "none"
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'GraphPassword',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select username,password FROM register")
            rows = cur.fetchall()
            for row in rows:
                if row[0] == username:
                    password = row[1]
                    break
        if password != 'none':
            output = '<center><img src="/static/password/'+password+'" alt="" width="400"
height="300"  id="myImgId" onmousemove="getPos(event)"/></center>'
            context= {'data':output}
            return render(request, 'ShowAuthenticateImage.html', context)
        if password == 'none':
            context= {'data':'Invalid username'}
            return render(request, 'Login.html', context)


def Register(request):
    if request.method == 'GET':
        return render(request, 'Register.html', {})

def Reset(request):
    if request.method == 'GET':
        return render(request, 'Reset.html', {})

def index(request):
    if request.method == 'GET':
        return render(request, 'index.html', {})
```

```
def Login(request):
    if request.method == 'GET':
        return render(request, 'Login.html', {})

def AdminLogin(request):
    if request.method == 'GET':
        return render(request, 'AdminLogin.html', {})


def getSpotValue(spot):
    arr = spot.split(",")
    values = []
    values.append(int(arr[0].strip()))
    values.append(int(arr[1].strip()))
    return values

def authspots(old_spot, new_spot):
    auth = False
    old_x = old_spot[0]
    old_y = old_spot[1]
    previous_x = old_x - 10
    forward_x = old_x + 10
    previous_y = old_y - 10
    forward_y = old_y + 10
    if new_spot[0] >= previous_x and new_spot[0] <= forward_x and new_spot[1] >= previous_y
and new_spot[1] <= forward_y:
        auth = True
    return auth

def PasswordAuthAction(request):
    if request.method == 'POST':
        global username
        spot1 = getSpotValue(request.POST.get('t1', False))
        spot2 = getSpotValue(request.POST.get('t2', False))
        spot3 = getSpotValue(request.POST.get('t3', False))
        spot4 = getSpotValue(request.POST.get('t4', False))
        password = "none"
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'GraphPassword',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select username,spot1,spot2,spot3,spot4 FROM register")
            rows = cur.fetchall()
            for row in rows:
                if row[0] == username:
```

```
                old_spot1 = getSpotValue(row[1])
                old_spot2 = getSpotValue(row[2])
                old_spot3 = getSpotValue(row[3])
                old_spot4 = getSpotValue(row[4])
                if authspots(old_spot1, spot1) and authspots(old_spot2, spot2) and
authspots(old_spot3, spot3) and authspots(old_spot4, spot4):
                    password = "success"
                    break
        if password == "success":
            context= {'data':'Welcome '+username+"<br/>Login successfull"}
            return render(request, 'UserScreen.html', context)
        else:
            context= {'data':'Invalid spot selection'}
            return render(request, 'Login.html', context)


def    PasswordAction(request):
    if request.method == 'POST':
        global username, password, contact, email, address
        spot1 = request.POST.get('t1', False)
        spot2 = request.POST.get('t2', False)
        spot3 = request.POST.get('t3', False)
        spot4 = request.POST.get('t4', False)
        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password =
'root', database = 'GraphPassword',charset='utf8')
        db_cursor  =  db_connection.cursor()
        student_sql_query = "INSERT INTO
register(username,password,contact,email,address,spot1,spot2,spot3,spot4)
VALUES("'+username+'","'+password+'","'+contact+'","'+email+'","'+address+'","'+spot1+'","'+
spot2+'","'+spot3+'","'+spot4+'")"
        db_cursor.execute(student_sql_query)
        db_connection.commit()
        print(db_cursor.rowcount, "Record Inserted")
        if db_cursor.rowcount == 1:
            context= {'data':'Signup Process Completed'}
            return render(request, 'Register.html', context)
        else:
            context= {'data':'Error in signup process'}
            return render(request, 'Register.html', context)

def RegisterAction(request):
    if request.method == 'POST':
        global username, password, contact, email, address
        username = request.POST.get('t1', False)
        contact = request.POST.get('t3', False)
        email = request.POST.get('t4', False)
```

```
        address = request.POST.get('t5', False)
        password = request.FILES['t2'].name
        myfile = request.FILES['t2']
        fs = FileSystemStorage()
        output = "none"
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'GraphPassword',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select username,email FROM register")
            rows = cur.fetchall()
            for row in rows:
                if row[0] == username:
                    output = username+" Username already exists"
                    break
                if row[1] == email:
                    output = email+" Email id already exists"
                    break
        if output == "none":
            fs.save('GraphPasswordApp/static/password/'+password, myfile)
            output = '<center><img src="/static/password/'+password+'" alt="" width="400"
height="300"  id="myImgId" onmousemove="getPos(event)"/></center>'
            context= {'data':output}
            return render(request, 'ShowImage.html', context)
        else:
            context= {'data':output}
            return render(request, 'Register.html', context)


def ViewUsers(request):
    if request.method == 'GET':
        output = '<table border=1 align=center width=100%>'
        font = '<font size="" color="black">'
        arr = ['Username','Password Image','Contact No','Email
ID','Address','Spot1','Spot2','Spot3','Spot4']
        output += "<tr>"
        for i in range(len(arr)):
            output += "<th>"+font+arr[i]+"</th>"
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'GraphPassword',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select * FROM register")
            rows = cur.fetchall()
            for row in rows:
                output += "<tr><td>"+font+row[0]+"</td>"
```

```
            output+='<td><img src=/static/password/'+row[1]+' height=100 width=100/></td>'
            output += "<td>"+font+row[2]+"</td>"
            output += "<td>"+font+row[3]+"</td>"
            output += "<td>"+font+row[4]+"</td>"

            output += "<td>"+font+row[5]+"</td>"
            output += "<td>"+font+row[6]+"</td>"
            output += "<td>"+font+row[7]+"</td>"
            output += "<td>"+font+row[8]+"</td>"
        context= {'data':output}
        return render(request, 'ViewUsers.html', context)

  def    UpdatePassword(request):
     if request.method == 'POST':
        global username, password
        spot1 = request.POST.get('t1', False)
        spot2 = request.POST.get('t2', False)
        spot3 = request.POST.get('t3', False)
        spot4 = request.POST.get('t4', False)
        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password =
'root', database = 'GraphPassword',charset='utf8')
        db_cursor    =    db_connection.cursor()
        student_sql_query = "update register set
password="'+password+'",spot1="'+spot1+'",spot2="'+spot2+'",spot3="'+spot3+'",spot4="'+spot
4+"' where username="'+username+'""
        db_cursor.execute(student_sql_query)
        db_connection.commit()
        print(db_cursor.rowcount, "Record Inserted")
        if db_cursor.rowcount == 1:
            context= {'data':'Password Reset Process Completed'}
            return render(request, 'Register.html', context)
        else:
            context= {'data':'Error in reset password'}
            return render(request, 'Reset.html', context)


def ResetAction(request):
    if request.method == 'POST':
        global username, password
        password = request.FILES['t1'].name
        myfile = request.FILES['t1']
        fs = FileSystemStorage()
        fs.save('GraphPasswordApp/static/password/'+password, myfile)
        output = '<center><img src="/static/password/'+password+'" alt="" width="400"
height="300"  id="myImgId" onmousemove="getPos(event)"/></center>'
        context= {'data':output}
```

```
        return render(request, 'ShowResetImage.html', context)

def AdminLoginAction(request):
    if request.method == 'POST':
        user = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        if user == 'admin' and password == 'admin':
            context= {'data':'Welcome '+user}
            return render(request, 'AdminScreen.html', context)
        else:
            context= {'data':'Invalid login'}
            return render(request, 'AdminLogin.html', context)
```

CHAPTER – 8

SCREENSHOTS AND RESULTS

3. RESULT

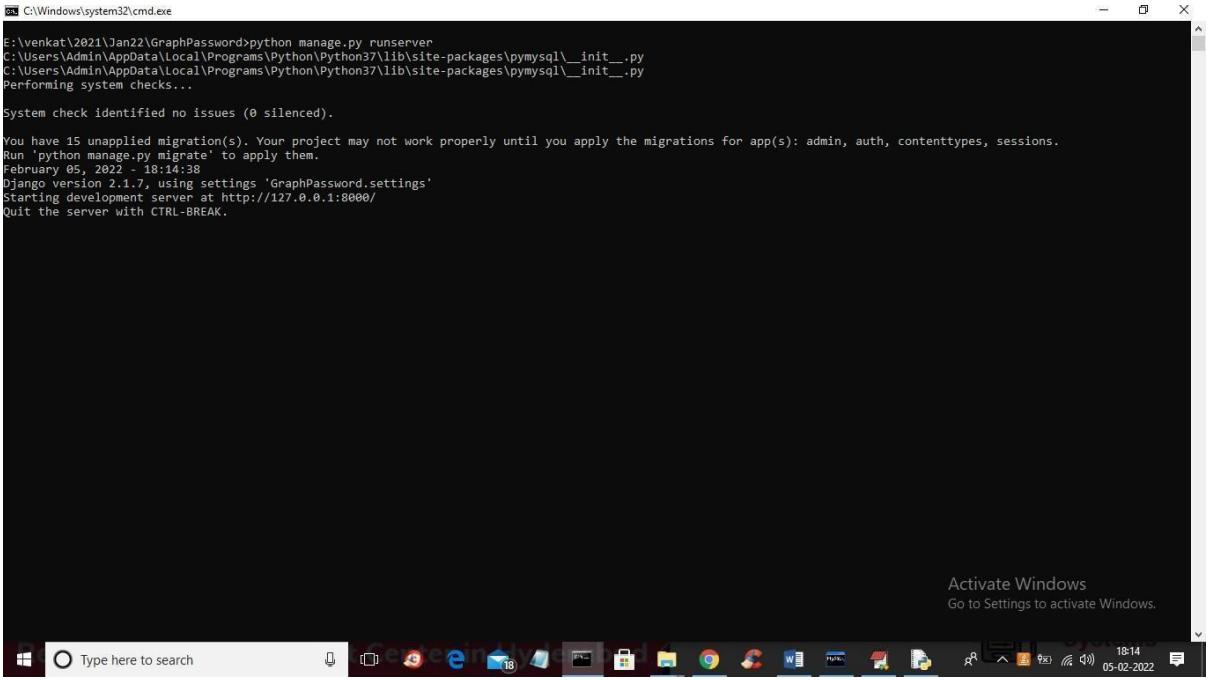
Web Based Graphical Password Authentication System
In this project we are authenticating users via images and this images will be uploaded at signup time and then ask user to click on image 4 times to select 4 different spots and user has to remember those points.
It's difficult for user's to select correct pixel X and Y location from mouse click so we provide region based authentication for example
If user select X = 120 and Y = 240 from mouse click then while authentication I deducted 10 pixels from X value and added 10 more pixels to X values which means
If user select X value between 110 to 130 and Y value between 230 and 250 then user get authenticated as actual or original X value 120 and Y value 240 falls between 110 to 130 and 230 to 250
To run project install python 3.7 and MYSQL and then copy content from DB.txt file and paste in MYSQL to create database
To implement this project we have designed following modules

1) Admin Login: using this module admin can login to application using username and password as admin and then after login can view all registered user details
2) New User Signup: using this module user can signup with the application and has to upload image in place of password and then select 4 spots and all this details will saved in database

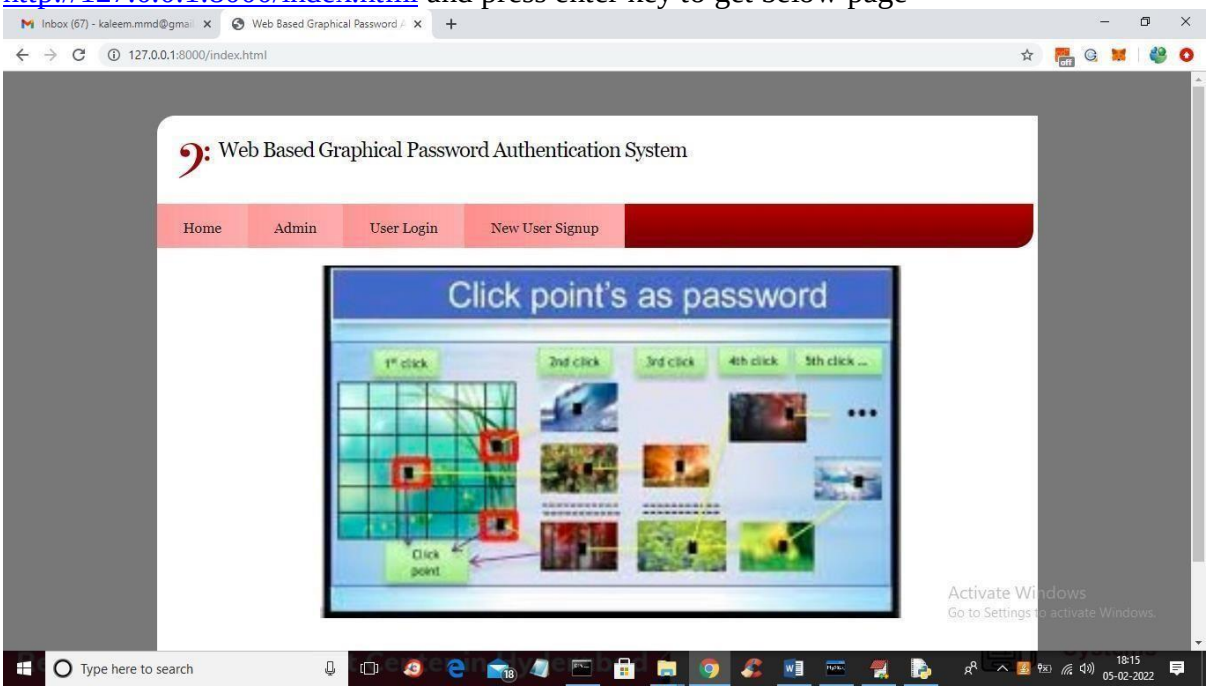
1) Reset Password: after login user can update password image and can enter new spots to reset password

SCREEN SHOTS

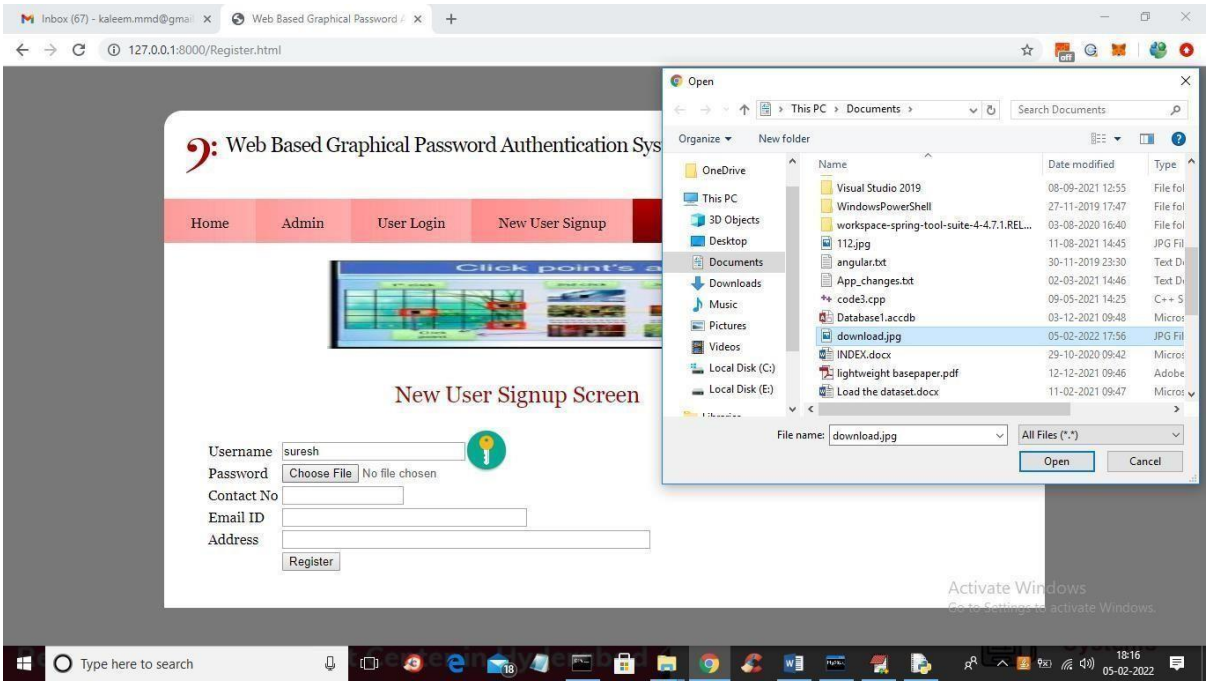
To run project double click on 'run.bat' file to start DJANGO server like below screen



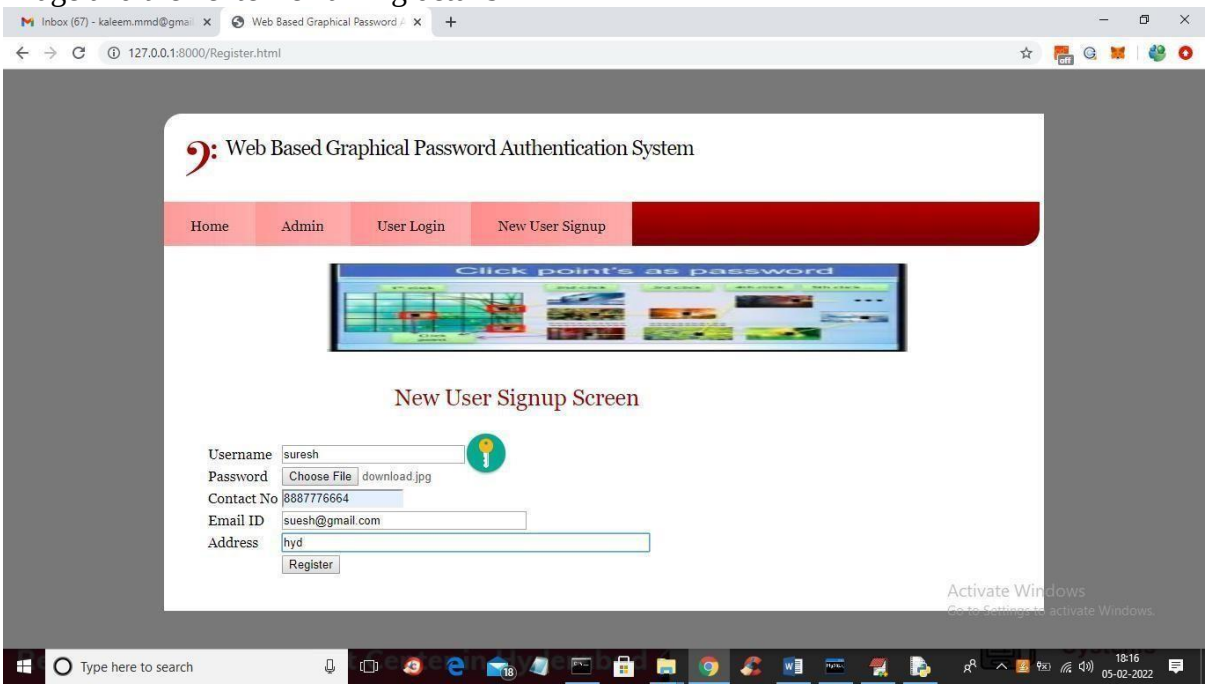
In above screen DJANGO server started and now open browser and enter URL as http://127.0.0.1:8000/index.html and press enter key to get below page



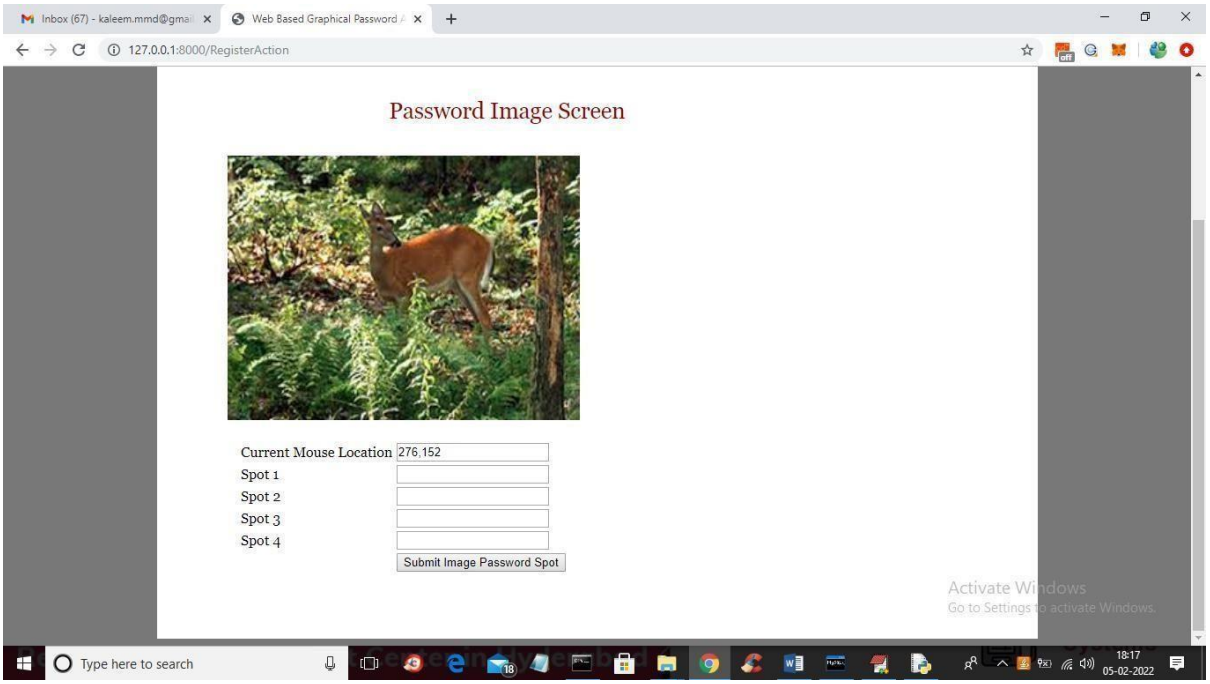
In above screen click on 'New User Signup' link to add new user details



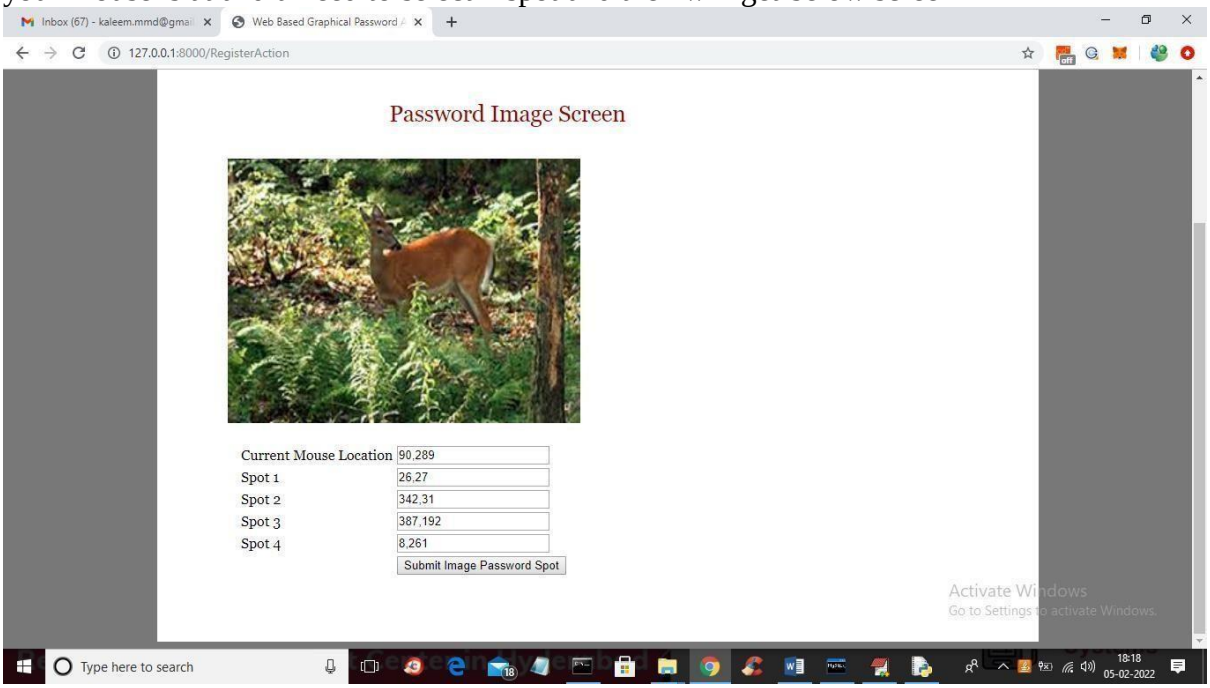
In above screen user is entering signup details and in place of password browsing and uploading image and then enter remaining details



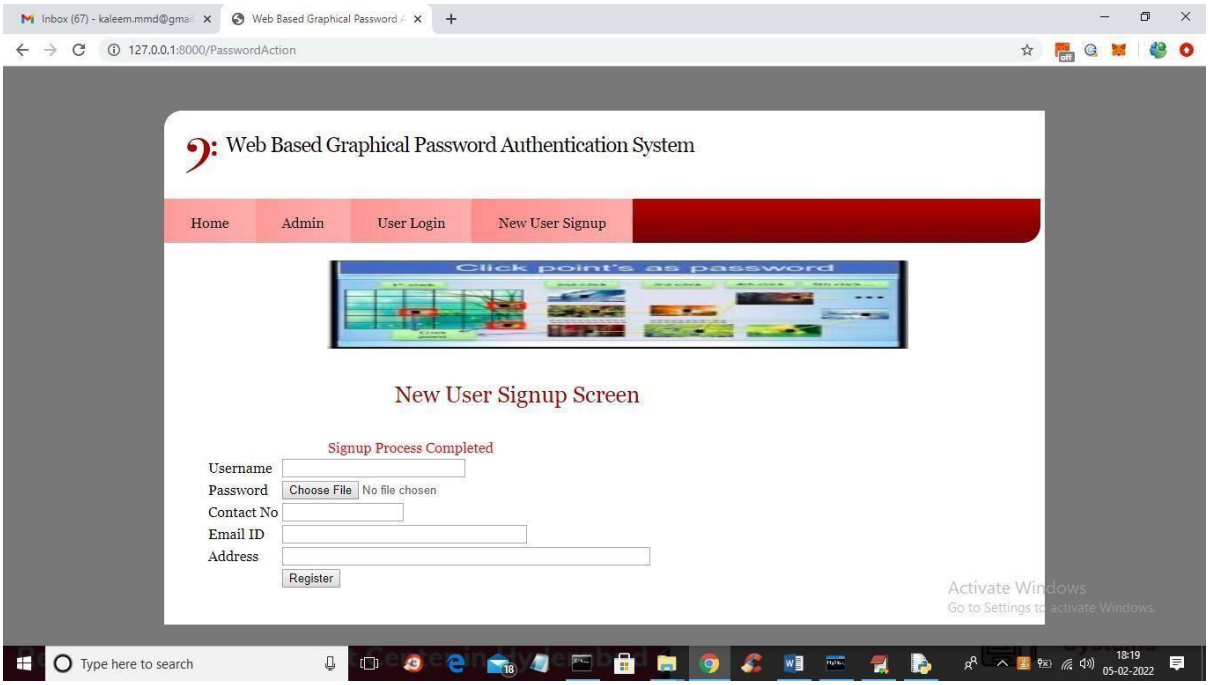
In above screen after entering all details click on ‘Register’ button to get below page with image



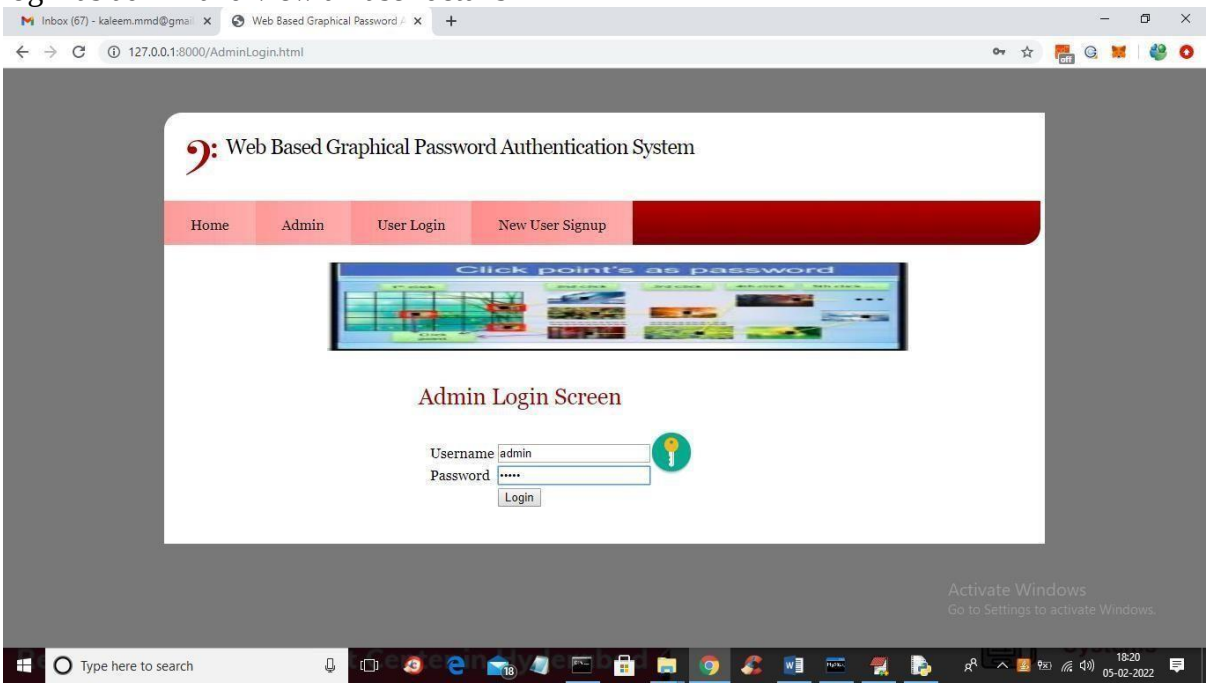
In above image user has to click on any part of the region then location will be added to text fields and first field will display the current location of mouse so you will know which point your mouse is at and u need to select 4 spot and then will get below screen



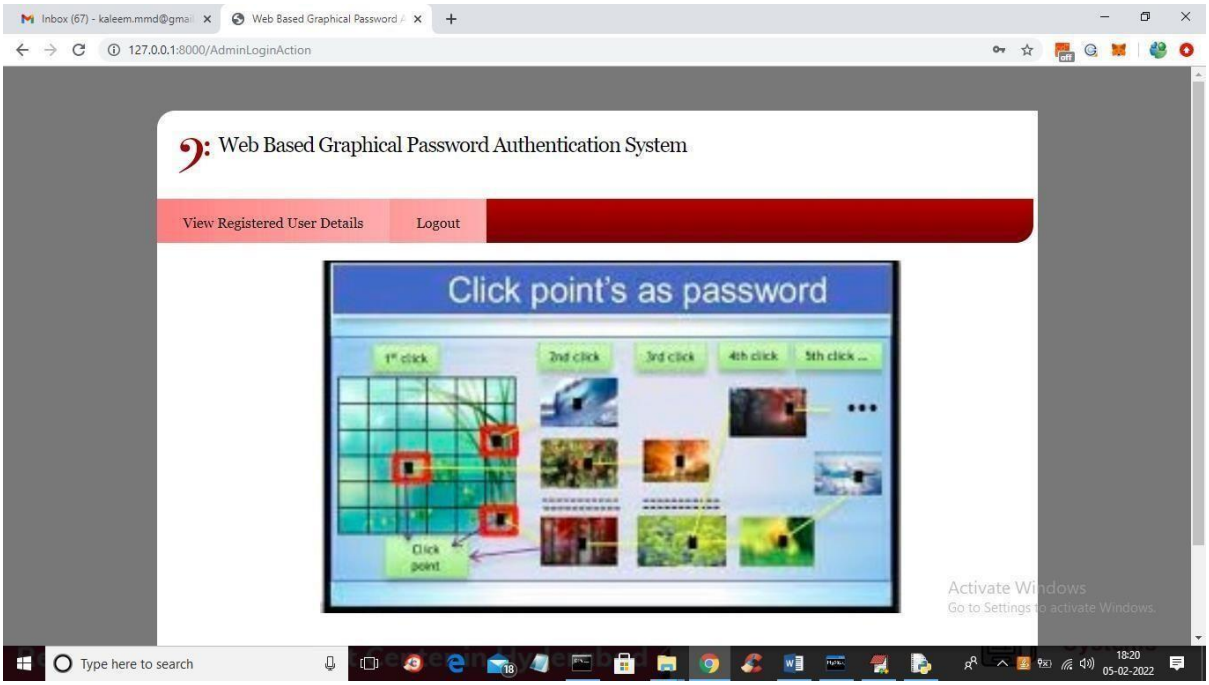
In above screen I selected 4 spots and all those X and Y selected values are filled in the text fields and then press button to get below page



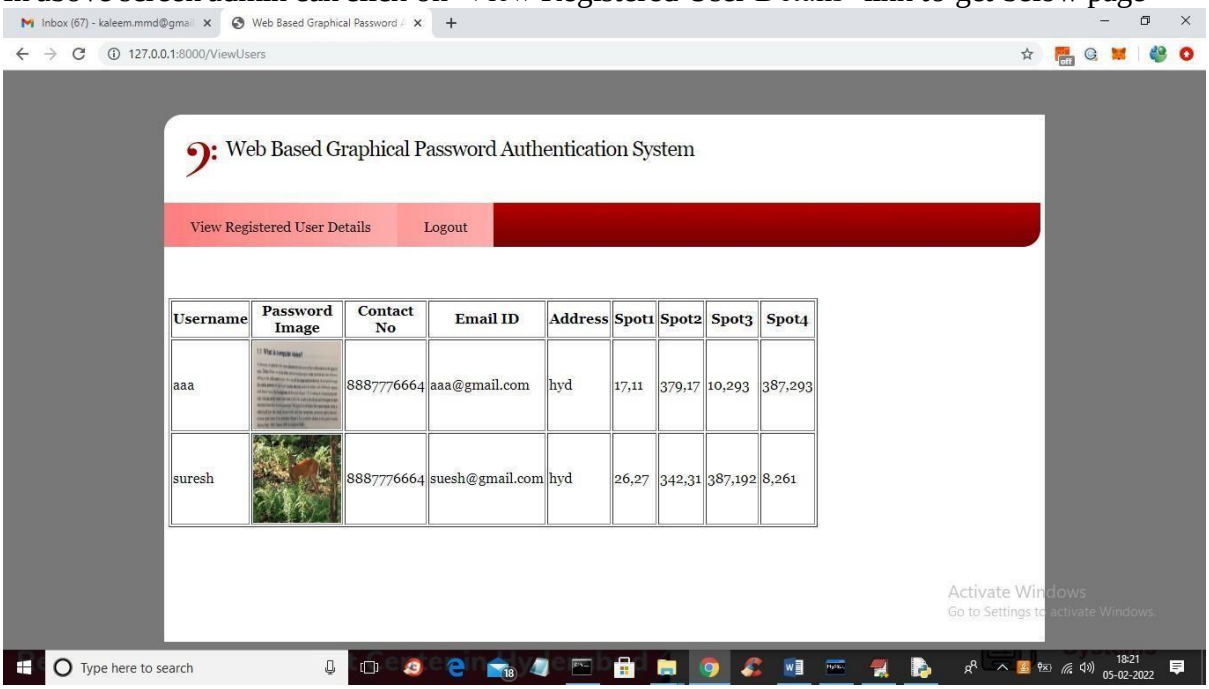
In above screen we got message as 'signup process completed' and now click on 'Admin' link to login as admin and view all user details



In above screen admin is login and after login will get below page

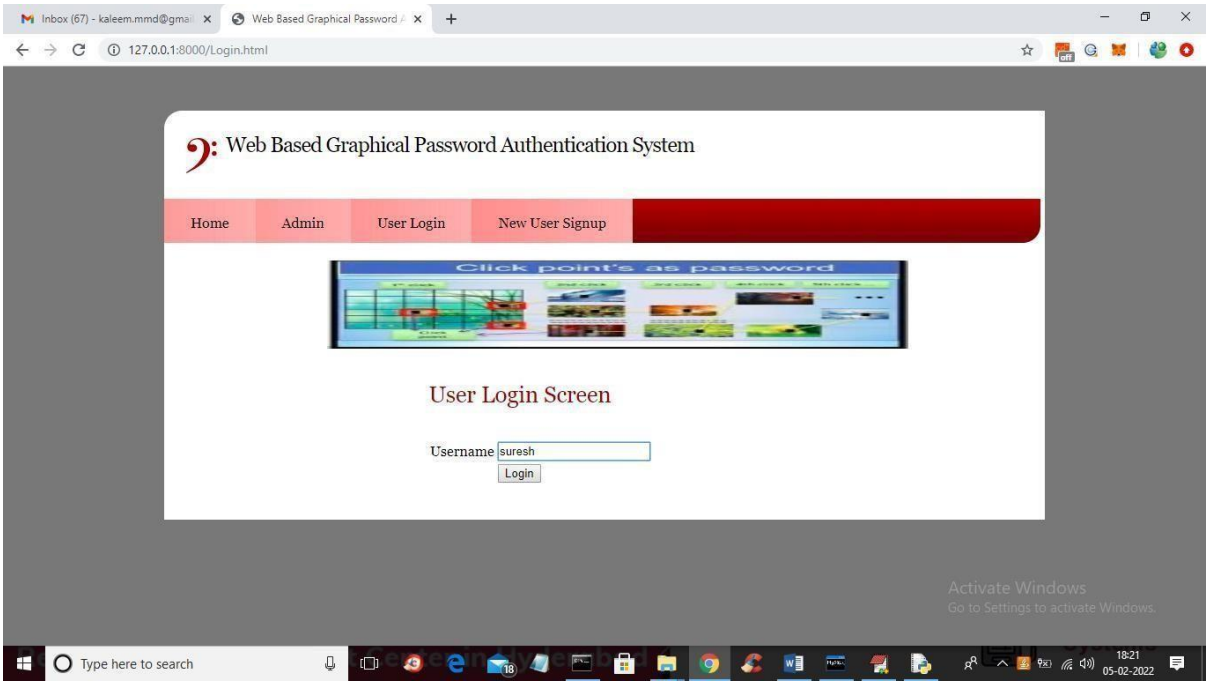


In above screen admin can click on 'View Registered User Details' link to get below page

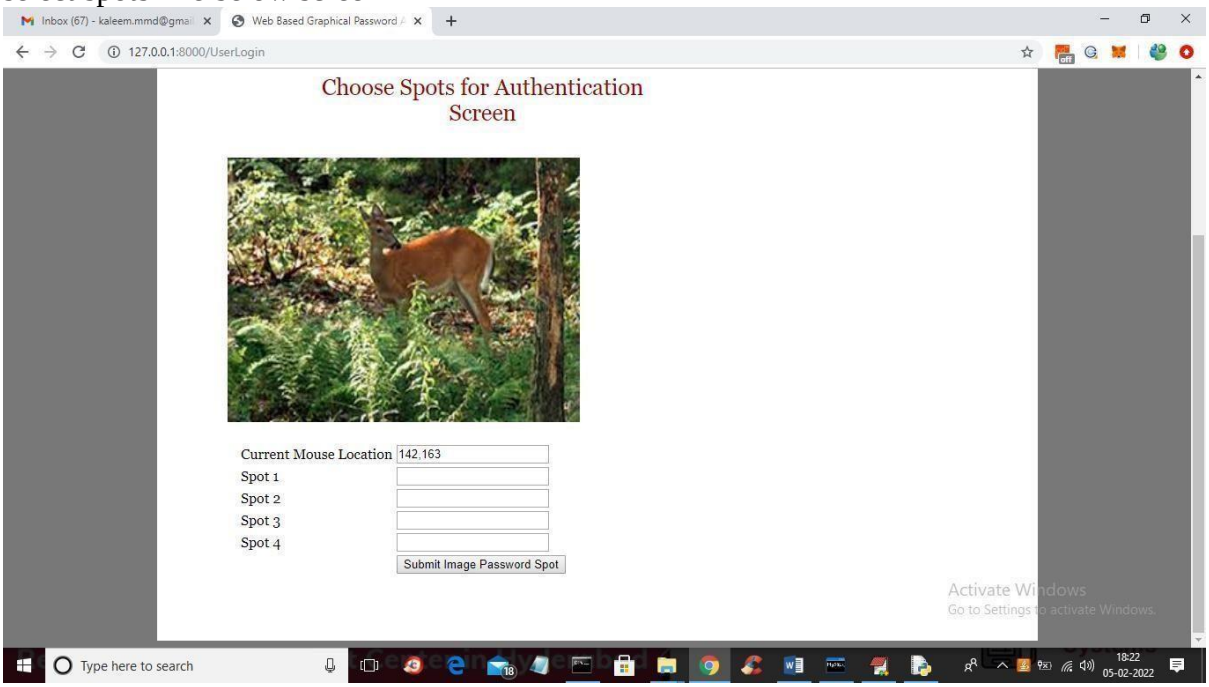


Username	Password Image	Contact No	Email ID	Address	Spot1	Spot2	Spot3	Spot4
aaa		8887776664	aaa@gmail.com	hyd	17,11	379,17	10,293	387,293
suresh		8887776664	suesh@gmail.com	hyd	26,27	342,31	387,192	8,261

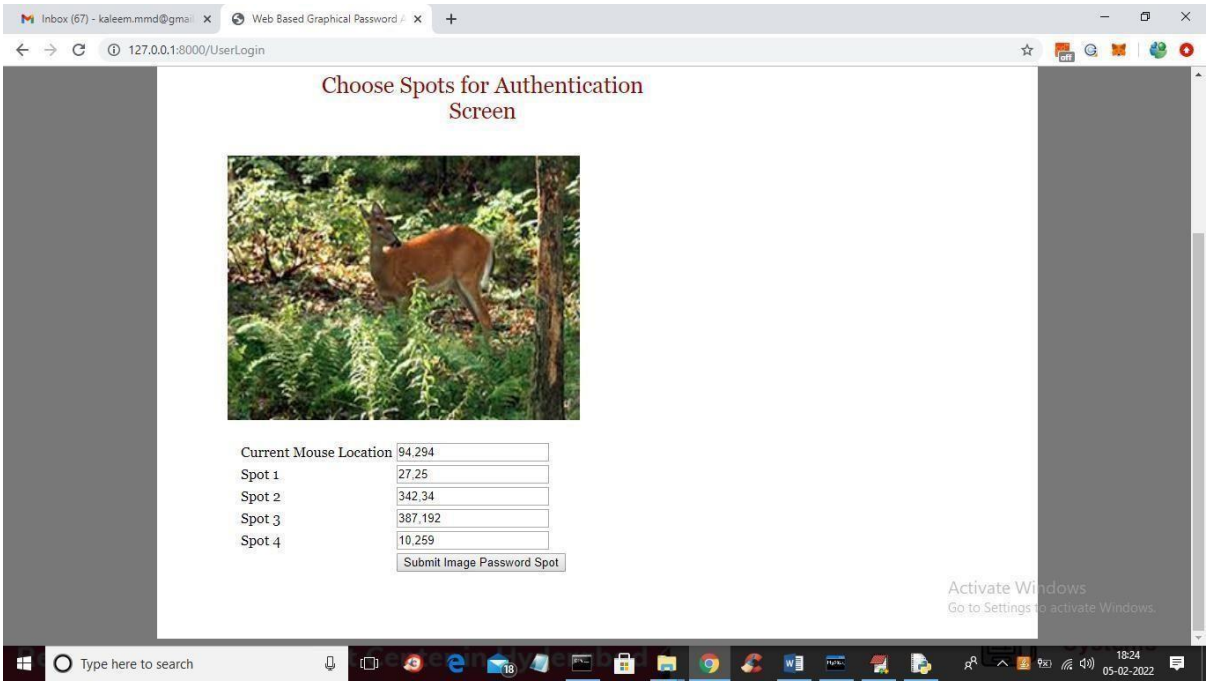
In above screen admin can see all users details such as username and password as image and selected 4 spots and now logout and login as user suresh



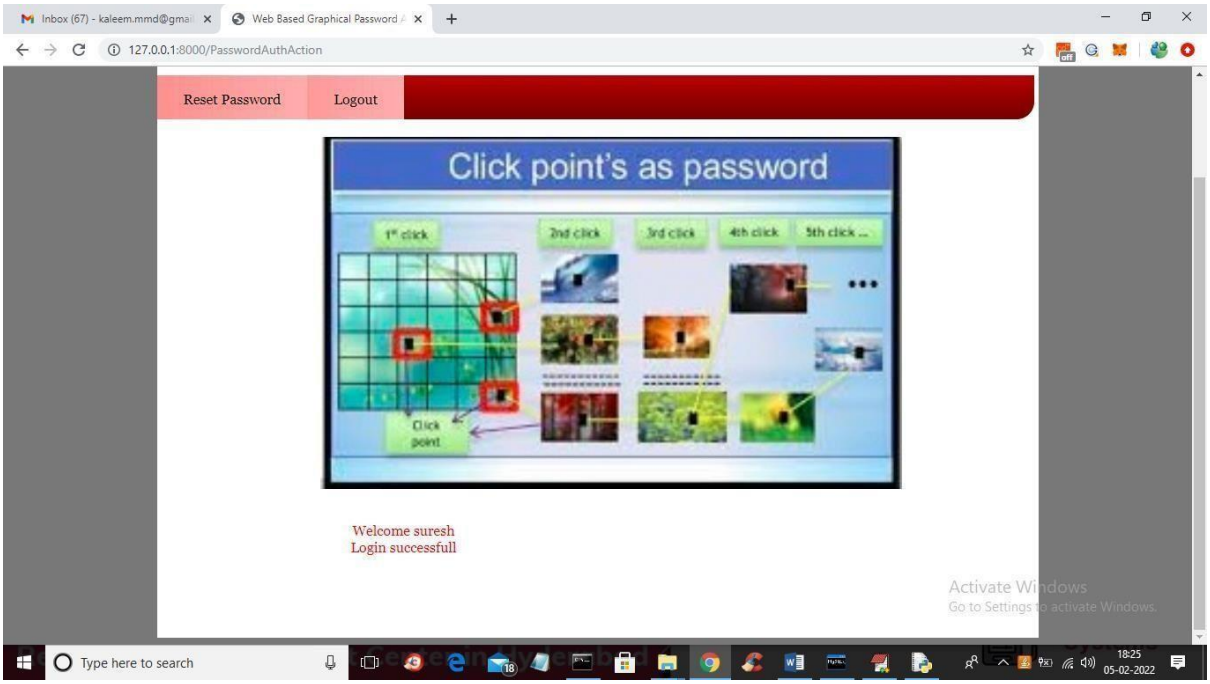
In above screen user is login by entering username and then press button to get image and to select spots like below screen



In above screen for login also user has to select 4 spots and then press button to authenticate



In above screen I selected values in range and then press button to get below screen and if given spots falls between correct password spots in database then user will get authenticated and get below screen



In above screen in red colour text we can see login is successful and if u give wrong details then authentication will get failed and if you want to reset password with new image and spots selection then click on 'Rest Password' link and repeat same steps.

NOTE: in any field if values are available then just delete those values from the field so new selected spot values can be appear

Fig. 5. **Home Page of Graphical PasswordAuthenticator**

The above image shows how the graphical password authenticator looks like. User can register an account from this home page and then can enter into his/her profile. The Registration section is secured with 2 layers of security. One is a textual password and another is a graphical one.

User can login to his/her profile by clicking the login button. The login page also includes 2 layers of security as mentioned above.

CHAPTER – 9

CONCLUSION

In the modern digital era, secure authentication mechanisms are essential to protect sensitive data and user privacy. Traditional text-based password systems, although widely used, suffer from several limitations such as weak password selection, password reuse, and vulnerability to attacks like brute force and shoulder surfing. These issues highlight the need for more secure and user-friendly authentication techniques.

In this project, a **Web-Based Graphical Password Authentication System** has been successfully designed and implemented to overcome the drawbacks of conventional authentication methods. The system introduces a graphical approach where users select specific points on an image as their password instead of relying solely on textual input.

The system uses a **tolerance-based matching mechanism**, which allows slight variations in user input during login. This improves usability without compromising security. The implementation includes essential modules such as user registration, login, password reset, and admin management.

One of the key advantages of the proposed system is its resistance to shoulder surfing attacks, as it is difficult to replicate exact click positions on an image. Additionally, graphical passwords are

easier to remember compared to complex text-based passwords, improving user experience. The use of modern technologies such as Python, Django, and MySQL ensures that the system is scalable, reliable, and easy to maintain.

Overall, the project successfully demonstrates that graphical password authentication is a practical and effective alternative to traditional methods. It provides enhanced security, better usability, and flexibility, making it suitable for real-world applications.

FUTURE SCOPE

Although the proposed graphical password authentication system provides improved security and usability, there are several opportunities for further enhancement and expansion. Future developments can focus on increasing the robustness, scalability, and adaptability of the system.

One possible enhancement is the implementation of **multi-image authentication**, where users select points across multiple images instead of a single image. This would significantly increase the complexity of the password and make it even more resistant to attacks. Another improvement could be the integration of **biometric authentication methods**, such as fingerprint or facial recognition, to create a multi-factor authentication system.

The system can also be enhanced by incorporating **advanced encryption techniques** for storing user credentials and graphical data. This would provide an additional layer of security and protect against database breaches. Additionally, implementing **machine learning algorithms** to detect unusual login patterns or suspicious activities can further strengthen the system's security.

Another area of improvement is the development of a **mobile-friendly version** of the system. With the increasing use of smartphones, optimizing the graphical authentication process for touch-based devices would enhance usability and accessibility. Features such as gesture-based authentication can also be explored.

The system can also be extended to support **cloud-based deployment**, allowing it to handle a large number of users efficiently. Integration with enterprise-level applications, banking systems, and e-commerce platforms can make the system more practical and widely applicable.

In conclusion, the proposed system lays a strong foundation for secure graphical authentication. With further enhancements and integration of advanced technologies, it has the potential to become a highly secure and widely adopted authentication solution in the future.

10. REFERENCES

1. Wantong zheng, Chunfu Jia, CombinedPWD: A New Password Authentication Mechanism Using Separators Between Keystrokes: 2017 13th International Conference on Computational Intelligence and Security (CIS)

2. Salisu Ibrahim Yusuf, Moussa Mahamat Boukar, User Define Time Based Change Pattern Dynamic Password AuthenticationScheme, 2018 14th InternationalConference on Electronics Computer

3. Yang Jingbo, Shen Pingping, A secure strong password authentiction protocol, 2010 2nd International Conference on Software Technology and Engineering